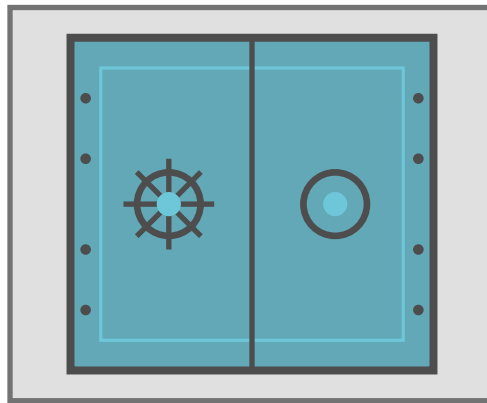


Criptografía y Seguridad

Seguridad en sistemas y aplicaciones

Políticas de seguridad y Control de acceso



Apunte integral — Teoría + Práctica

Clase 6 · Capítulos 4–8, *Computer Security: Art and Science* (M. Bishop)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de la transcripción de la clase teórica (Prof. Pablo Abad) y de las diapositivas oficiales de la teórica y de la práctica.

Índice

1. Definiendo seguridad en aplicaciones	2
1.1. La tríada CIA	2
2. Conceptos de control de acceso	3
2.1. La tríada desde el punto de vista de los accesos	4
2.2. Identidad del sujeto	4
3. Políticas vs. Mecanismos	4
4. Modelos de seguridad	5
5. Modelo Bell–LaPadula (confidencialidad)	5
5.1. Reglas (principios)	5
5.2. Compartimentos, reticulado y dominancia	7
6. Modelo Biba (integridad)	8
6.1. Variante 1: Biba estricto	9
6.2. Variante 2: Biba Ring-Policy	10
6.3. Variante 3: Biba Low-Watermark	10
7. Modelo Clark–Wilson (integridad comercial)	10
7.1. Reglas formales (clase práctica)	11
8. Más allá de los modelos teóricos	12
9. Mecanismos de control de acceso: MAC vs. DAC	13
10. Control discrecional por extensión	14
10.1. Matriz de control de accesos	14
10.2. Listas de control de acceso (ACL)	14
10.2.1. Conflictos en ACL	15
10.2.2. Derechos por defecto (clase práctica)	15
10.3. Usuarios privilegiados, transferencias y revocaciones	15
11. Control discrecional por comprensión	16
11.1. RBAC — control basado en roles	16
11.2. ABAC — control basado en atributos	16
11.3. RuBAC — control basado en reglas	17
12. Resolución de múltiples reglas	17
13. Reglas basadas en contexto	18
13.1. Zero-Trust y Open Policy Agent (OPA)	18
14. Ejemplos reales de control de acceso	19
14.1. Unix / Linux file system	19
14.2. SELinux (Security-Enhanced Linux)	20
14.3. Windows file system	20
14.4. PostgreSQL: roles y políticas	20
14.5. Squid Web Proxy	21
14.6. AWS (IAM)	21
14.7. API gateway	21

15.OAuth 2.0 (Open Authorization)	21
15.1. El flujo de autorización	21
15.2. Refresh Token	22
15.3. Scope (alcance) — un mecanismo DAC	22
Lectura recomendada	23

1 Definiendo seguridad en aplicaciones

En criptografía aprendimos algo fundamental: **no hay una única definición de seguridad**. Para una misma función criptográfica podía haber distintas pruebas, y cada prueba era, en el fondo, *una definición distinta de seguridad*. Al pasar de funciones matemáticas a **sistemas dinámicos**, ocurre lo mismo: la seguridad depende del contexto (aplicación, servicio, empresa).

Política de seguridad — la definición operativa

Existe una definición genérica para sistemas complejos: si pudiésemos clasificar todos los **estados** de un sistema en *seguros* e *inseguros*, entonces

un sistema es seguro mientras sus estados queden dentro de la política de seguridad, es decir, mientras transicione únicamente entre estados que consideramos seguros.

La **política de seguridad** es, literalmente, *la definición de qué estados de un sistema vamos a considerar seguros* frente al resto.

Esta definición es más teórica que práctica —si modelamos una aplicación como una máquina de estados, la cantidad de estados es *abismalmente grande* y difícil de clasificar—, pero aporta dos conceptos muy poderosos que sí aplican siempre:

1. Necesitamos una forma de **caracterizar**, sacando “una foto en el tiempo”, si el estado actual del sistema es seguro o no. *Eso es la política de seguridad.*
2. Un sistema que pasa **eventualmente** a un estado inseguro **deja de ser seguro**, *aunque después vuelva* a un estado que consideremos seguro.

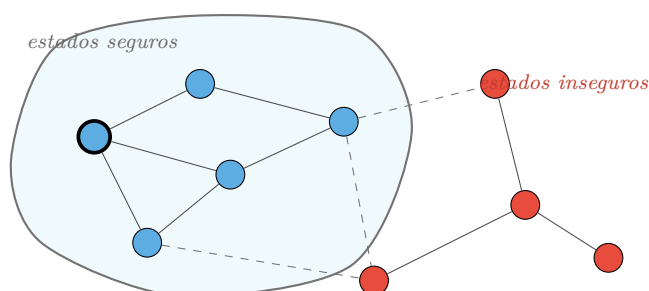


Figura 1: Mientras el sistema transita por estados *dentro* de la región (azules), es seguro. Si una transición lo lleva a un estado inseguro (rojo), deja de ser seguro. (*Recreación de la diapositiva 2.*)

El costo de volver a un estado seguro

El punto 2 tiene enorme importancia en seguridad defensiva. Si a un servidor se le **comprometió la seguridad**, las buenas prácticas dicen que habría que “prenderle fuego y traer uno nuevo” (o, sin tanto drama físico, **reinstalarlo**). Es muy difícil, una vez comprometida, volver a un estado del que se tenga garantía. Por eso trabajar en seguridad requiere cierto *mindset* de adaptabilidad y resiliencia: como en criptografía, no se puede bajar la probabilidad de incidente a cero, y cuando hay un incidente el costo de recuperación es altísimo.

1.1 La tríada CIA

La seguridad puede enfocarse desde varios lados. En criptografía vimos **confidencialidad** y **integridad**; el espectro completo incorpora un tercer servicio, la **disponibilidad**. A estos tres se los conoce como la **tríada CIA** (*Confidentiality, Integrity, Availability*).

Tríada CIA

- **Confidencialidad:** atañe a la *diseminación* de la información. Hay información que debe ser accesible a cierto grupo y no a otro. (*Guardar la información en secreto: acceso solo a los autorizados.*)
- **Integridad:** tiene que ver con relaciones de *confianza*. Necesito poder certificar la calidad de algo: que una transacción ocurrió, que los datos son correctos, o que el *origen* es quien dice ser. Se distingue la **integridad de datos** de la **integridad de origen**.
- **Disponibilidad:** capacidad de acceder a la información o ejecutar una acción *en el momento* en que se la necesita. Involucra **temporalidad**, es más dinámica y por eso no se estudia tanto en criptografía (que es estática).

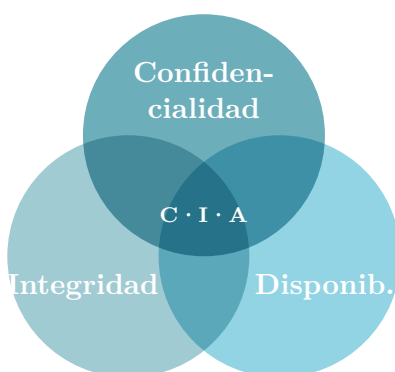


Figura 2: La tríada CIA. La seguridad de un activo se preserva sosteniendo las tres propiedades. (*Recreación de la diapositiva 3.*)

2 Conceptos de control de acceso

Para construir teoría sobre seguridad se introducen tres conceptos básicos, engañosamente simples, que son la base de todo:

Sujeto, Objeto y Acceso

Sujeto

La entidad que es *origen* de algún estímulo: ejecuta acciones sobre otras entidades. Pueden ser personas, servicios, aplicaciones, servidores.

Objeto

El *destino* de esos estímulos. Generalmente recursos: datos, servicios, bases de datos, tablas, programas, archivos.

Acceso

Las *formas de interacción* entre un sujeto y un objeto (las operaciones posibles: leer, escribir, borrar, mover, ejecutar, ...).

El ejemplo de siempre: Linux

En Linux (paradigma Unix) el esquema es directo: los **usuarios** son los sujetos, los **archivos** son los objetos (“todo es un archivo”) y los **permisos** son los accesos que rigen qué operaciones se pueden hacer. Es un modelo *engañosamente simple*: todos interactuamos con uno así, pero

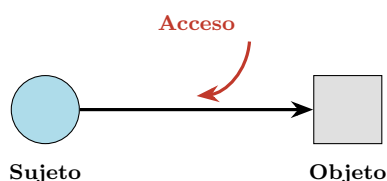


Figura 3: Un sujeto interactúa con un objeto mediante un acceso. (*Recreación de la diapositiva 5.*)

esconde mucha complejidad (lo retomamos en la sección de ejemplos).

2.1 La tríada desde el punto de vista de los accesos

La importancia de la tríada *sujeto–objeto–acceso* es que permite reducir los tres servicios de seguridad a una mínima expresión modelable:

Las políticas de seguridad se implementan controlando cómo los sujetos acceden a los objetos.

- **Confidencialidad:** existen sujetos que NO pueden acceder a ciertos objetos.
- **Integridad:** existen objetos que solo pueden ser modificados por ciertos sujetos confiables.
- **Disponibilidad:** existen objetos que pueden ser accedidos por ciertos sujetos cuando lo requieran.

2.2 Identidad del sujeto

La **identidad** es lo que **identifica unívocamente al sujeto** dentro del universo de posibles sujetos. Es un *requerimiento para la autenticación* (tema de la clase siguiente). Ejemplos: en Windows/Active Directory el SID (S-1-5-21-...) y el DistinguishedName; en Linux el UID (la primera columna numérica de /etc/passwd, p. ej. root:x:0:0:...).

3 Políticas vs. Mecanismos

Distinción clave (clase práctica)

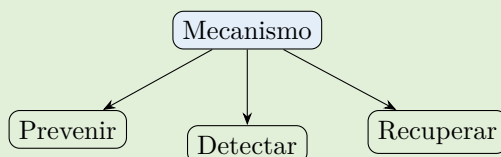
Políticas

Definen **qué acciones son seguras** (permitidas) y **qué acciones son inseguras** (prohibidas).

Mecanismos

Procedimiento, herramienta o método para **garantizar el cumplimiento** de la política.

Los mecanismos cumplen tres funciones complementarias:



En síntesis: la **política** dice qué se considera seguro; el **mecanismo** es el cómo se hace cumplir.

4 Modelos de seguridad

Tenemos la idea de sujeto–objeto y la de servicios (confidencialidad, etc.), pero ante un problema práctico necesitamos resolver *cómo* definir qué estados son seguros.

¿Qué es un modelo de seguridad?

Un **modelo de seguridad** es un **modelo formal y lógico** que define las **reglas e interacciones** que permiten implementar un mecanismo de control de acceso. A través de una sustentación formal (matemática) se demuestra que el sistema es seguro.

- Están **especializados**: hay modelos de *confidencialidad*, de *integridad*, e incluso de *disponibilidad*.
- Son **generalizaciones** que ya sabemos que funcionan: evitan *reinventar la rueda* y razonar a un nivel más alto (analogía: el modelo es a un lenguaje de alto nivel lo que la solución artesanal es al ensamblador).
- **No alcanzan por sí solos** en un sistema real: se aplican a *partes* del sistema.

Hay muchísimos modelos, pero dos o tres dominan el escenario y conviene conocerlos: **Bell–LaPadula** (confidencialidad), **Biba** (integridad) y **Clark–Wilson** (integridad en entornos comerciales).

5 Modelo Bell–LaPadula (confidencialidad)

Bell y LaPadula (investigadores de IBM) diseñaron este modelo, célebre por ser el **primer modelo de políticas de confidencialidad aprobado para uso en organismos gubernamentales de EE.UU.** y el primero *público* utilizable en ambientes civiles (análogo, en su impacto, a lo que fue DES). Está pensado para el **ámbito militar**, focalizado en proteger la **confidencialidad**.

Objetivo y elementos

Objetivo: prevenir el *acceso no autorizado* a la información; las modificaciones no autorizadas son *secundarias*. Combina **acceso mandatorio** y **acceso discrecional**. Elementos:

- Sujetos y Objetos.
- Modos de acceso = $\{\text{read}, \text{write}, \dots\}$.
- Clasificación de seguridad = $\{\text{TS}, \text{S}, \text{C}, \text{U} \dots\}$ (*Top Secret, Secret, Confidential, Unclassified*).
- Niveles de seguridad = Clasificación $\times$ Categoría (reticulado / *lattice*).

La información **debe clasificarse** según su nivel de confidencialidad, y todos los sujetos deben tener un **nivel de acceso** definido, con los mismos posibles niveles que la información.

5.1 Reglas (principios)

El modelo tiene dos reglas, más una tercera que sintetiza a las dos. Para enunciarlas con generalidad usamos la **relación de dominancia dom** y notamos $L(s)$ y $L(o)$ los niveles del sujeto s y del objeto o .

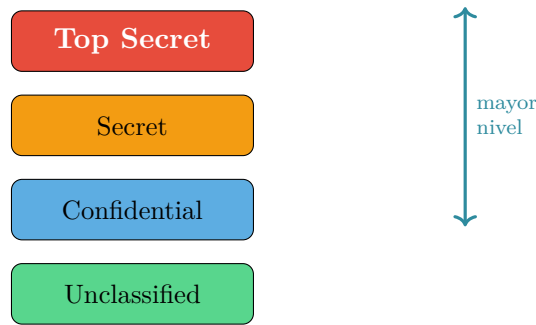


Figura 4: Niveles de clasificación del modelo original. (*Recreación de la diapositiva 11.*)

Las tres reglas de Bell–LaPadula

1. **Condición de seguridad simple — *Read Down*.** El sujeto s puede *leer* el objeto o si y solo si

$$L(s) \text{ dom } L(o) \quad \text{y } s \text{ tiene permiso para leer } o.$$

Un sujeto puede leer objetos de su nivel *hacia abajo*.

2. **Propiedad * (estrella) — *Write Up*.** El sujeto s puede *escribir* el objeto o si y solo si

$$L(o) \text{ dom } L(s) \quad \text{y } s \text{ tiene permiso para escribir } o.$$

Un sujeto solo puede escribir hacia su nivel *o más arriba*.

3. **Propiedad estrella fuerte (síntesis).** Si un sujeto necesita *leer y escribir* el mismo objeto, solo podrá hacerlo con objetos de su **misma clasificación** (es lo único que satisface ambas reglas a la vez).

Regla mnemotécnica: *Read down, Write up*. Pensar “escritura” y “lectura” como operaciones que *cambian* o *no cambian* el estado: leer puede ser ver contenido, metadata, tamaño o existencia; escribir puede ser modificar, borrar, agregar.

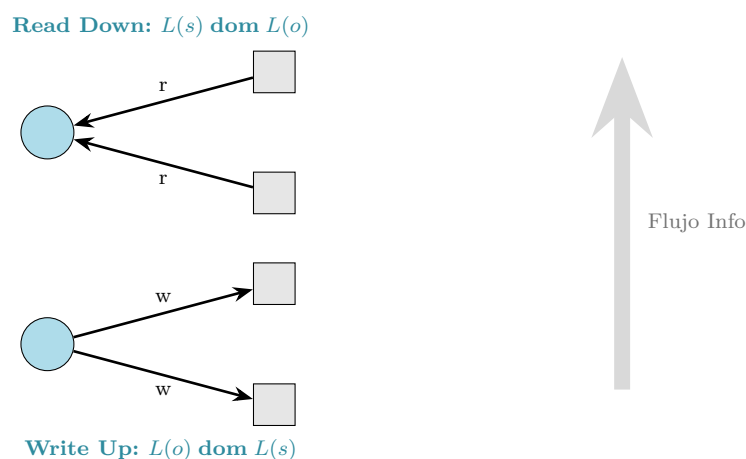


Figura 5: Un sujeto lee hacia abajo y escribe hacia arriba; la información “sube”. (*Recreación de las diapositivas de teórica y práctica.*)

¿Por qué la regla de escritura? El control de flujo de información

La primera regla resuelve el **acceso directo**: si clasifico un documento como *Top Secret* y solo yo y mi socio somos TS, nadie más debería leerlo. Pero supongamos que la escritura fuese libre. Abro el documento en un editor que, por las dudas, guarda una *copia temporal* en otro directorio. Ese nuevo archivo tiene la misma información pero *otras reglas*: ¿quién garantiza que no lo lea alguien que no debe? La primera regla sola no impide que alguien con acceso “vuelque” la información en otro lado. La **regla de cierre (*write up*)** ataca eso: si tenés acceso a información muy sensible, *no podés escribir hacia niveles más bajos*, porque estarías filtrándola. Bell–LaPadula no garantiza confidencialidad estática, sino un **control de flujo de información**: hay demostraciones formales (sobre máquinas de Turing, en el libro de Bishop) de que *no existe ningún camino* por el que la información “baje” de nivel.

El extremo: el Proyecto Manhattan

El modelo es **muy estricto**: “si accediste a este tipo de información, ya no podés hablar con el resto de los mortales”. Esto importa en inteligencia militar, donde una fuga puede costar vidas. Incluso *sin querer*, quien conoce información de alto nivel puede filtrarla parcialmente (una respuesta tangencial, un gesto). La aplicación de esta regla llevó a que, durante el desarrollo de la bomba atómica, EE.UU. **aislara físicamente a los científicos del Proyecto Manhattan en un pueblo entero** junto con sus familias, para que no interactuaran con gente fuera del proyecto: estaban aplicando Bell–LaPadula.

Limitación: el principio de tranquilidad

El mayor problema de Bell–LaPadula es el **principio de tranquilidad**: asume que la clasificación de sujetos y objetos *no cambia con el tiempo*. Pero en la realidad sí cambia: los empleados ascienden o son despedidos, y la información hoy confidencial puede ser pública en unos meses. Las implementaciones reales agregan **mecanismos al costado para reclasificar**. Un síntoma típico: los *emails* de gente de organismos gubernamentales llevan al pie una leyenda de *desclasificación* (“*declassified for use at unclassified level*”), porque según las reglas estrictas un sujeto con acceso *secret* no debería poder interactuar en foros abiertos. El modelo **Clark–Wilson** se usa como complemento para arbitrar estas transiciones.

5.2 Compartimentos, reticulado y dominancia

Solo cuatro niveles son demasiado simples para una organización real. El modelo agrega una **lista de etiquetas** (categorías) que se asignan a sujetos y objetos; un **nivel + lista de etiquetas** forma un **compartimento**. Esto implementa el concepto de **need-to-know**: *un general a cargo de criptografía no tiene por qué conocer los secretos de los misiles, sin importar su nivel de acceso*.

Relación de dominancia

Con compartimentos, el “mayor/menor” se generaliza a la relación de **dominancia**. Para niveles (L, C) y (L', C') —donde C es el conjunto de etiquetas (categorías)—:

$$(L, C) \text{ dom } (L', C') \iff L' \leq L \wedge C' \subseteq C.$$

Las reglas quedan: s puede *leer* o si $L(s) \text{ dom } L(o)$, y puede *escribir* o si $L(o) \text{ dom } L(s)$. Ejemplos:

$$\begin{aligned} (\text{TS}, \{\text{nuclear}\}) \text{ dom } (\text{S}, \{\text{nuclear}\}), \quad & (\text{S}, \{\text{nuclear}\}) \text{ dom } (\text{S}, \emptyset), \\ (\text{TS}, \{\text{nuclear}, \text{crypto}\}) \text{ dom } (\text{TS}, \{\text{nuclear}\}). \end{aligned}$$

Esto forma un **reticulado** (*lattice*), un **conjunto de orden parcial**: no todos los pares se pueden ordenar. Por ejemplo, $(S, \{\text{nuclear}\})$ y $(S, \{\text{crypto}\})$ *no* se dominan en ningún sentido (mismo nivel, pero ninguna lista de etiquetas es subconjunto de la otra) $\Rightarrow$ están **efectivamente separados**: ni lectura ni escritura entre ellos.

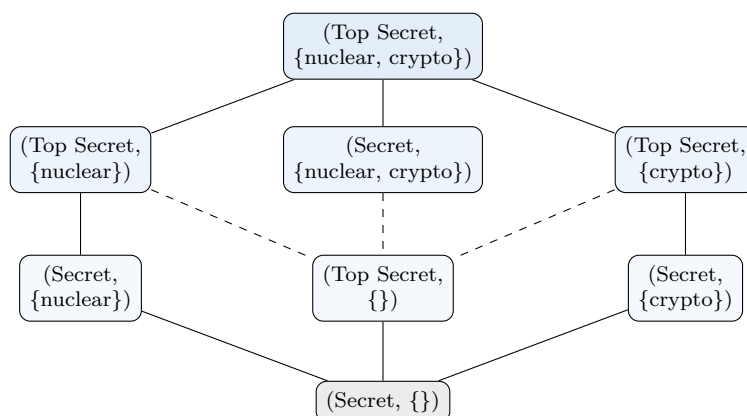


Figura 6: Reticulado (orden parcial) de compartimentos. Arriba domina a abajo; ramas sin camino no se dominan. (*Recreación de la diapositiva 13.*)

Implementación y uso real

La implementación es **simple**: por cada objeto se guarda como metadata un *nivel* y una *lista de etiquetas*; por cada sujeto (usuario), lo mismo. Resolver una operación es aplicar las dos reglas. Por su rigidez, **rara vez se aplica a todo un sistema**: se reserva para lo muy crítico. Caso típico en empresas: separar por etiquetas *finanzas* / *producción* / *marketing*, con gente que accede solo a una y gente que accede a varias. La compartimentalización tipo Bell-LaPadula aparece, p. ej., en sistemas que manejan **información médica** sujetos a regulación.

6 Modelo Biba (integridad)

Biba desarrolló una **familia de unos 7 modelos** (en conjunto, “modelo Biba”). Es famoso por ser el primer modelo público que *no* buscaba confidencialidad sino proteger la **integridad** de la información. Tiene importancia en sistemas **documentales** o de **evidencia**, donde se necesitan garantías fuertes de *procedencia* y de *no adulteración*: registros contables/financieros que hay que retener, sistemas judiciales, etc.

Objetivo y elementos

Objetivo: preservar los datos y su **integridad**. Identifica las maneras *autorizadas* en que la información puede ser alterada y *quiénes* están autorizados a hacerlo. Elementos: Sujetos, Objetos, Modos de acceso $\{r, w, \dots\}$ y **niveles de integridad** $il(\cdot)$, donde *a mayor nivel, mayor confianza*.

Los niveles suelen interpretarse mejor por *calidad/confianza* que por *secreto*: *no confiable*, *poco confiable*, *confiable*, *muy confiable*, *fidedigna*. Es estructuralmente **el inverso** de Bell-LaPadula en cuanto a reglas.

Una persona es tan confiable como las fuentes que usa

La intuición del modelo: para considerar confiable a un sujeto, necesito que la información en la que basa lo que hace o dice (lo que *lee*) sea de *ese nivel o más alto*. Y la información que un sujeto *genera* (escribe) será tan buena como el valor que le atribuyo a ese sujeto: nunca mayor. De ahí salen las dos reglas.

6.1 Variante 1: Biba estricto

Reglas de Biba estricto

1. **Escritura — Write Down.** *s* puede *modificar o* si y solo si

$$\text{il}(s) \geq \text{il}(o).$$

2. **Lectura — Read Up.** *s* puede *observar o* si y solo si

$$\text{il}(o) \geq \text{il}(s).$$

Regla mnemotécnica: *Read up, Write down* (la información “baja” de nivel).

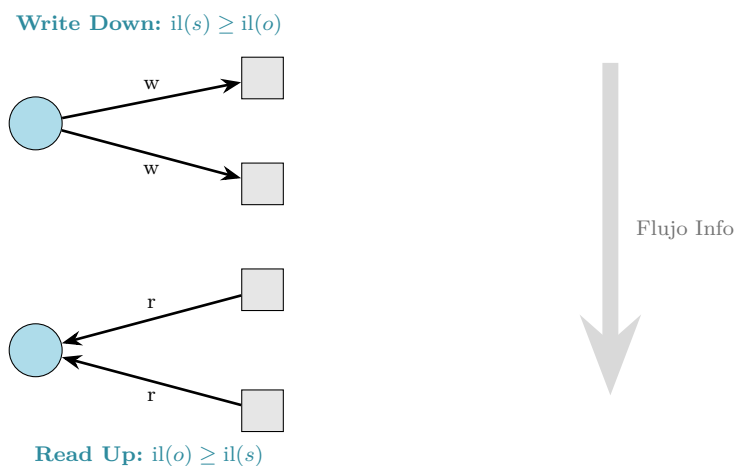


Figura 7: Biba estricto: se lee hacia arriba y se escribe hacia abajo. (*Recreación de las diapositivas de práctica.*)

Usos del modelo estricto

Se usa en el **curado de información**: muchos diarios lo aplican “sin saberlo” al sanitizar y verificar información para generar artículos confiables. También aplica si el **sujeto es un programa** y los **objetos son datos**: un sistema cerrado bien clasificado evita que un programa poco robusto procese datos de mala calidad (p. ej. sin protección contra inyección SQL). *Filosofía detrás del aviso del SO*: cuando bajás un ejecutable de Internet, el sistema te advierte “viene de una fuente dudosa, ¿seguro querés correrlo?”; si lo bajás del App Store, no pregunta nada. Está clasificando la confianza del archivo por su origen: es una aplicación de la idea de Biba.

6.2 Variante 2: Biba Ring-Policy

Reglas de Biba Ring-Policy

1. **Escritura** — *Write Down*: igual que el estricto, $il(s) \geq il(o)$.
2. **Lectura**: *cualquier* sujeto s puede leer *cualquier* objeto o (no se restringe la lectura).

6.3 Variante 3: Biba Low-Watermark

Refleja cierto **dinamismo** (a diferencia de los estáticos). Filosofía: *uno es tan confiable como la información que usa*.

Regla de Biba Low-Watermark

1. **Escritura** — *Write Down*: $il(s) \geq il(o)$.
2. **Lectura**: cualquier sujeto puede leer cualquier objeto, **pero** al hacerlo su nivel de integridad se *reclasifica al mínimo*:

$$\text{si } s \text{ lee } o : \quad il(s) = \min(il(s), il(o)).$$

Sistemas “adaptativos”

Si el sistema me considera muy íntegro y en algún momento interactúo con algo de baja integridad, mi nivel **baja automáticamente**: tendré que volver a pedir permiso o revalidarme para recuperar el nivel anterior. Esta es la base de muchos sistemas que hoy se venden como *adaptativos* / *personalizados*: según las interacciones, van cambiando el nivel de confianza del sujeto.

7 Modelo Clark–Wilson (integridad comercial)

Creado por David D. Clark y David R. Wilson, es un modelo de **integridad** más acotado y simple, pero introduce un concepto clave para los **sistemas operativos**. Está pensado para escenarios donde conviven, al mismo tiempo, **información y usuarios confiables y no confiables**.

Objetivo y elementos

Objetivo: integridad, especialmente en **entornos comerciales**. Elementos:

- **Sujetos y Objetos**, y estos últimos se dividen en:
 - **CDI** (*Constrained Data Items*): datos restringidos / protegidos.
 - **UDI** (*Unconstrained Data Items*): datos no restringidos.
- **Procedimientos** (administran accesos y permisos):
 - **TP** (*Transformation Procedure*): cambia los ítems de un estado válido (consistente) a otro estado válido.
 - **IVP** (*Integrity Verification Procedure*): verifica que los ítems estén en un estado válido.

- Un usuario **no puede modificar un CDI en forma directa**: debe hacerlo a través de un **TP**.
- Un usuario **sí** puede acceder a datos **UDI** sin un TP.
- Es responsabilidad de los TP **validar la integridad** de la información.
- Incorpora *separation of duty* (separación de tareas): *quien modifica los datos no puede ser el mismo que verifica la integridad*.

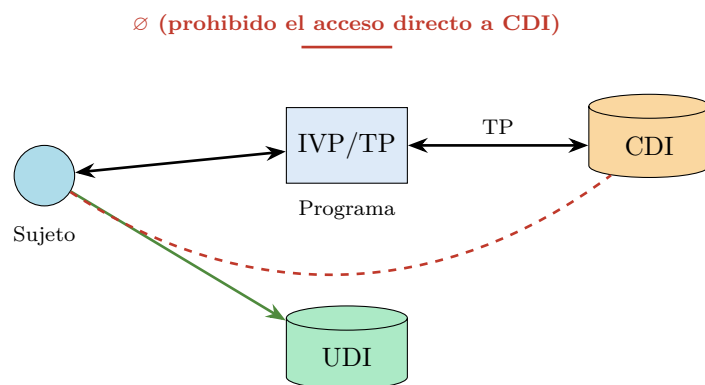


Figura 8: El sujeto modifica un CDI *solo* a través de un TP (con verificación IVP); a los UDI accede directamente. (*Recreación de la diapositiva de práctica.*)

7.1 Reglas formales (clase práctica)

Las reglas se dividen en dos grupos:

1. Reglas de Certificación (CR) — discrecionales, las aplica el Administrador

C1 (Certificación)

Los IVP aseguran que los CDI están en un estado válido.

C2 (Validez)

Un TP toma un estado válido y lo lleva a otro estado válido, sobre un conjunto certificado $\{TP_i, (CDI_a, CDI_b, \dots)\}$.

C3 (Separación de tareas)

Las *relaciones certificadas* que asocian un conjunto de CDI con un TP cumplen la separación de tareas.

C4 (Journal certification)

En un CDI tipo bitácora, los TP guardan información para reconstruir la operación.

C5 (Validación de UDI)

Los TP validan los UDI: o convierten el UDI en un CDI válido, o el UDI es rechazado.

2. Reglas de Aplicación (ER) — mandatorias, las aplica el Sistema

E1 (Aplicación de validez)

El sistema mantiene la lista de relaciones certificadas: TP_f opera sobre $CDI_o \iff (f, o) \in C$.

E2 (Aplicación de separación)

El sistema mantiene la lista de relaciones permitidas $(UserID, TP_i, \{CDI_a, \dots\})$.

E3 (Identificación de usuarios)

El sistema debe *autenticar* a quien intenta ejecutar el TP.

E4 (Cambios)

Las autorizaciones solo las puede cambiar el administrador (*security officer*).

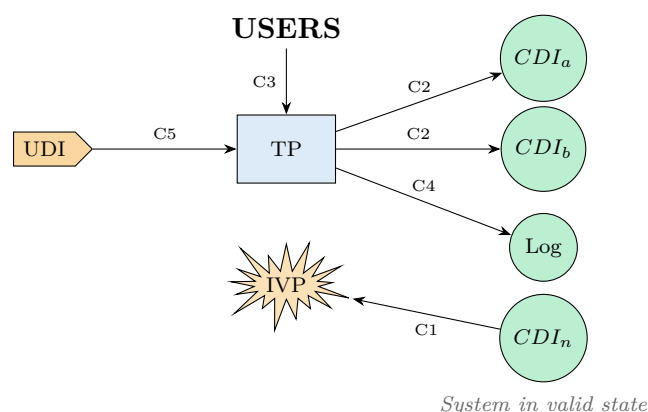


Figura 9: Reglas de certificación: el TP lleva los CDI de un estado válido a otro, el IVP los verifica, los UDI se validan y todo queda registrado. (*Recreación de la diapositiva “Clark–Wilson (Formal)”.*)

De los bancos al kernel: separation of duty en acción

El modelo se aplica también a sistemas *no* informáticos. Es la base del **principio de separación de privilegios**. En un banco, al abrir una cuenta de empresa se arma un *registro de firmas*: por debajo de cierto monto bastan ciertas firmas; por encima se exige una firma más. Para procesar una transacción siempre hay *dos partes*: una **verifica** las firmas y otra, **distinta, ejecuta** la transacción —aunque todo sea automático—.

En sistemas operativos, esta idea derivó en la separación **user space / kernel space**: el *kernel space* es un ambiente de alta confianza (puede saltarse todos los controles: acceso directo al disco, etc.); el resto corre como *no confiable* en *user space*. ¿Cómo se cruza de uno a otro? Con las **syscalls** (llamadas al sistema): el usuario arma un *dataset* describiendo qué quiere (“abrir tal archivo para lectura, con tal *file descriptor*...”) y dispara la syscall. Un proceso **transforma** (TP) esa información llevándola a la zona del kernel, otro **verifica** su integridad (IVP) (que el *file descriptor* sea un número positivo, etc.) y, recién entonces, un programa privilegiado ejecuta. Es una aplicación directa de Clark–Wilson para proteger la integridad del sistema operativo.

8 Más allá de los modelos teóricos

Los tres modelos vistos son **teóricos**: en la práctica *no se aplican directamente* a un sistema completo, pero son la base de muchas implementaciones. Es raro encontrar el nombre explícito de Bell–LaPadula / Biba / Clark–Wilson, porque mucha gente aplica la idea sin saber el nombre. Aun así, conviene saber dónde aparecen:

Modelo	¿Qué sistemas se basan en él?
Bell–LaPadula	La organización por <i>compartimentos</i> aparece en sistemas que manejan información médica (sujetos a regulación).
Biba	Sistemas de <i>clasificación y archivo</i> de información para medios de prensa, legajos judiciales o declaraciones fiscales .
Clark–Wilson	El concepto de <i>kernel space</i> / <i>user space</i> de muchos sistemas operativos (p. ej. Unix).

Lectura recomendada sobre los modelos (papers originales)

- Bell76.pdf — documento original del modelo Bell–LaPadula.
- Biba75.pdf — documento original del modelo Biba.
- A_Comparison_of_commercial_and_military.pdf — documento original del modelo Clark–Wilson.

9 Mecanismos de control de acceso: MAC vs. DAC

El **control de acceso** es un mecanismo de seguridad que **regula quién (sujeto) puede acceder a qué (objeto) y bajo qué condiciones**.

Los modelos suelen ser muy restrictivos, así que rara vez cubren todo un sistema; en el resto necesitamos manejar el control de acceso con dos grandes familias:

MAC — Mandatorio	DAC — Discrecional
<i>Basado en reglas</i>	<i>Basado en identidad</i>
Control que se aplica en forma obligatoria a todos los accesos de los sujetos a los objetos. Reglas universales (p. ej. Bell–LaPadula Lattice). Las políticas se definen para el sistema .	La asignación y modificación de permisos queda a discreción del sujeto responsable del objeto (o de un administrador). Las políticas se definen para el usuario .
Aplica sobre: todos los accesos, los cambios en las reglas/atributos, y el intercambio de información entre sujetos.	Típico de repositorios donde los sujetos crean los objetos (file systems, object storage). Permite distribuir la responsabilidad.

¿Por qué existe el DAC?

Los sistemas mandatorios son demasiado rígidos para sistemas de *propósito general* (un sistema operativo, una base de datos, Google Workspace o Microsoft 365). En el DAC las propiedades de seguridad *no* las da el sistema, sino el administrador según cómo lo configure: es una forma elegante de cubrir múltiples escenarios y de no tener que dar garantías fuertes. La contracara: las garantías dependen de *cómo* se configura.

10 Control discrecional por extensión

Dentro del DAC, los accesos se pueden definir **por extensión** (enumerando) o **por comprensión** (mediante reglas/propiedades). Empecemos por extensión.

10.1 Matriz de control de accesos

Representación matricial: en un eje los **sujetos** (filas), en el otro los **objetos** (columnas); en cada intersección, las **operaciones permitidas** para esa combinación.

		ACL (columnas)			
		Obj1	Obj2	...	ObjN
Capability lists (filas)	Usuario1	r	rw		rwX
	Usuario2	rx	r		rx
	...				

Figura 10: Matriz de accesos. Sus *columnas* son las **ACL** (por objeto); sus *filas* son las **capability lists** (por sujeto). (*Recreación de las diapositivas de práctica.*)

Inconvenientes de la matriz

Es simple de entender pero, como implementación directa, tiene problemas serios. Sirve más como *concepto ilustrativo*:

- **Escalabilidad:** con 100 000 usuarios y 1 000 000 de objetos crece de forma violenta.
- **Eficiencia:** la búsqueda puede consumir mucho tiempo y recursos.
- **Mantenimiento:** agregar un sujeto obliga a definir su relación con *todos* los objetos (y viceversa). Propenso a errores.
- **Almacenamiento y limitaciones dinámicas:** no se adapta bien a entornos donde cambian permisos, sujetos u objetos.

Clave (consulta de Joaquín): para la *computadora* tener una matriz gigante no es problema; el problema es el *pobre humano* que con su discreción tiene que poner los valores. *Eso* es lo que no escala.

10.2 Listas de control de acceso (ACL)

En un sistema grande los permisos aparecen como “islas”: la matriz queda **dispersa** (casi todo vacío). Una forma eficiente de representarla es, por cada objeto, enumerar solo los pares (*sujeto, permisos*) que tienen sentido. Eso son las **ACL**: son las *columnas* de la matriz.

Definición formal de ACL

$$\text{ACL}(o) = \{ (s_i, p_i) \mid s_i \in \mathcal{S} \wedge p_i \subseteq \mathcal{R} \}$$

con $\mathcal{S}$ = sujetos, $\mathcal{O}$ = objetos, $\mathcal{R}$ = permisos. Extensiones habituales:

- **Denegar por defecto:** si $s \notin \text{ACL}(o) \Rightarrow s$ no tiene permiso sobre o .
- **Grupos:** $G = \{s_1, s_2, \dots, s_i \mid s_i \in \mathcal{S}\}$, p. ej. $\text{ACL}(o) = \{(s_1, r), (G, rw), \dots\}$. Permite

tratar a muchos sujetos como una sola entrada y aplicar reglas a nivel grupo.

- **Wildcards:** $ACL(o) = \{(s_1, r), (s_2, *), \dots\}$.
- **Permisos por defecto** (*defaults*) y **patrones** para identificar objetos: `/users/*`, `/home/**/*.*tmp`.

10.2.1 Conflictos en ACL

Si **más de una entrada** de la ACL otorga permisos a un sujeto s , hay que resolver:

1. **Permitir** el acceso si *alguna* entrada lo da.
2. **Grant All:** denegar si *alguna* entrada lo deniega (exige que *todas* den acceso).
3. **First Rule / First Match:** aplicar la *primera* regla encontrada.

Ejercicio resuelto — conflictos de ACL

Sea $s_1 \in G$ y

$$ACL(o) = \{(G, rw), (s_1, w), (\{s_1, s_2\}, wx), \dots\}.$$

¿Qué puede hacer s_1 según cada política?

Política	Entradas que aplican	Resultado para s_1
Permitir (unión)	rw, w, wx	r, w, x (la unión)
Grant All (intersección)	$rw \cap w \cap wx$	w
First Rule	primera: (G, rw)	r y w

10.2.2 Derechos por defecto (clase práctica)

Otras reglas aplicables cuando hay una entrada específica y un *default*:

1. **Override / Best match:** si el sujeto tiene una entrada específica, usarla; si no, usar el *default*.
2. **Augment:** partir del *default* y **agregarle** las entradas explícitas.

Ejercicio resuelto — override vs. augment

Sea $ACL(o) = \{(s_1, x), (*, r), \dots\}$ (el *default* “*” da r a todos). ¿Qué puede hacer s_1 ?

- **Override:** usa la entrada específica $\Rightarrow$ puede x .
- **Augment:** default + específica $\Rightarrow$ puede r (porque todos pueden) x .

10.3 Usuarios privilegiados, transferencias y revocaciones

Operaciones sobre las ACL (clase práctica)

Usuarios privilegiados 1. *Root* o administrador.

2. *Owner:* puede modificar la ACL del objeto sobre el que tiene el derecho “own”.

Transferencia

Cambia el sujeto de una entrada: el sujeto 1 *deja* de tener los derechos, que pasan al sujeto 2.

Delegación

El sujeto 1 le *delega* derechos al sujeto 2 **sin perderlos**.

Revocación

El *owner* quita la entrada o quita derechos. Si hubo delegación, se produce **revocación en cascada**.

11 Control discrecional por comprensión

Para sistemas grandes no alcanza con la representación dispersa: en cada “isla” hay grupos de usuarios y objetos con casi la misma distribución de accesos. La solución es definir el control **por comprensión** (mediante *reglas*/propiedades) en vez de por extensión, guardando reglas compactas que podrían reconstruir la matriz *bajo demanda*. El criterio de agrupación define el mecanismo:

Sigla	Por...	Idea
RBAC	roles	<i>Role-Based Access Control.</i>
ABAC	atributos	<i>Attribute-Based Access Control.</i>
RuBAC	reglas	<i>Rule-Based Access Control.</i>

No hay uno mejor: según la situación conviene uno u otro.

11.1 RBAC — control basado en roles

Estereotipa **roles** (más abarcativo que un grupo): genera “plantillas” que agrupan permisos. Ejemplos de roles en una empresa: *account manager*, persona de ventas, administrador de IT, administrador de operaciones, usuario común; o *gestor de pacientes*, *analista de compras*, *DBA*.

- Los **permisos se asignan a los roles**; los roles se asignan a los sujetos (o a grupos).
- Considera **jerarquías de roles**. En sistemas tradicionales un usuario tiene un solo rol, pero las aplicaciones complejas permiten *varios*.
- Una vez definidos los roles, la **gestión diaria es simple**: a un usuario nuevo se le dan los roles correctos.

¿RBAC es MAC o DAC?

Algunos libros lo describen como **mandatorio** (porque define “reglas de juego universales”), pero es una visión sesgada. Hay una **discrecionalidad fuerte**: alguien tiene que *asignar* usuarios a roles, y las garantías dependen de cómo se haga esa asignación —característica que define a lo discrecional—. Además, cuando un usuario puede tener **varios roles**, aparece un problema interesante: *múltiples sistemas de control de acceso interactuando sobre la misma operación* (p. ej. un esquema de roles más una zona protegida por Bell–LaPadula), lo que obliga a resolver conflictos.

11.2 ABAC — control basado en atributos

Las reglas se basan en **atributos**: del sujeto (rol, edad...), del objeto (tipo, clasificación de seguridad...), de la acción y del contexto (tiempo, ubicación...).

11.3 RuBAC — control basado en reglas

Muy parecido a roles, pero **sin asignación de sujetos**. Una regla en su forma más básica contiene: **Principal**, **Objeto**, **Tipo de acceso** y si está **permitido o bloqueado**. Típicamente hay una **lista ordenada de reglas** y “aplica la primera que ocurra” (u otra política de resolución).

12 Resolución de múltiples reglas

Cuando varias reglas aplican de forma **contradictoria**, se usa alguna política de resolución. **El mismo juego de reglas resuelve muy distinto según cómo se manejen los conflictos** (clave si alguna vez configuran firewalls):

Políticas de resolución

- **First match:** las reglas tienen un orden; aplica la *primera* que hace *match*.
- **Best match:** aplica la regla *más específica* (las reglas más específicas priman sobre las generales).
- **Deny over Allow:** cualquier regla que *deniega* explícitamente tiene precedencia sobre las que permiten (deben permitir *todas* las que apliquen).

Ejemplo: reglas de Firewall

Un *firewall* arbitra el tráfico entre dos redes. En su forma más simple es un sistema de reglas ordenadas. Considérese:

#	Proto	Source IP	Dest IP	Port	Action
1	TCP	10.1.1.1	20.1.1.1	80	Accept
2	TCP	10.1.1.2	20.1.1.1	80	Deny
3	TCP	10.1.1.0/24	20.1.1.1	80	Deny
4	TCP	10.1.1.3	20.1.1.1	80	Accept
5	TCP	10.2.2.0/24	20.2.2.5	80	Deny
6	TCP	10.2.2.5	20.2.2.0/24	80	Deny
7	TCP	10.3.3.0/24	20.3.3.9	80	Accept
8	TCP	10.3.3.9	20.3.3.0/24	80	Deny
9	IP	0.0.0.0/0	0.0.0.0/0	0-65535	Deny

Caso A — paquete 10.1.1.1 ⇒ 20.1.1.1:80 (hacen *match* las reglas 1 y 3):

- *First match* ⇒ regla 1 ⇒ **Accept**.
- *Deny over Allow* ⇒ la 3 deniega ⇒ **Deny**.

Caso B — paquete 10.3.3.9 ⇒ 20.3.3.9:80 (hacen *match* las reglas 7 —red /24— y 8 —host específico—):

- *Best match* ⇒ la 8 es más específica ⇒ **Deny**.

Observación: en *First match*, estas reglas dicen “no permitas comunicación hacia la red 10.1.1.x *salvo* desde el host 10.1.1.1”; para expresar lo mismo con *Best match* hay que escribirlas distinto (“permití a toda la red *salvo* a tal host”). La regla 9 final (0.0.0.0/0 Deny) implementa el *denegar por defecto*.

13 Reglas basadas en contexto

Permiten construir reglas que incluyen **información de contexto**, no solo la entrada/ salida de la operación. El **contexto** lo forman todos los datos asociados al sujeto, el objeto, el momento o el lugar del acceso —elementos que cambian *independientemente* de las reglas—:

- Momento del día; ubicación geográfica del origen de la conexión.
- Departamento al que pertenece la persona; si el usuario estaba de vacaciones.
- Nivel de confiabilidad de la máquina (p. ej. del DBA).

Permiten detectar **situaciones anómalas**. Ejemplos:

- Un usuario se loguea desde China y desde Argentina con 5 minutos de diferencia.
- Una usuaria que figura de licencia por maternidad se loguea en una base de datos del *datacenter*.
- Un usuario inicia dos VPNs en menos de 5 minutos con dos IPs distintas.

El nicho: prevención de fraude

Estas reglas son raras (muy sofisticadas) pero tienen un nicho claro: la **prevención de fraude** en e-commerce y sistemas financieros. Distinción importante:

- Un **problema de seguridad** es un abuso que se *saltea* las reglas del sistema.
- Un **fraude** *sigue* las reglas del sistema, pero hace algo que el sistema no esperaba (p. ej. pagar con una tarjeta de crédito robada: el pago “sale bien”, pero el dueño reclamará y se anulará).

El control anti-fraude estima la *probabilidad* de que una operación sea fraudulenta (en el login, el checkout o el panel de administración) y la deniega ante indicios.

13.1 Zero-Trust y Open Policy Agent (OPA)

La lógica de estas reglas suele requerir **programación**, y muchos sistemas no se pueden modificar. En el modelo **Zero-Trust**, la evaluación de contexto se define como una **entidad externa** al *Policy Engine*: un tercero valida cada operación.

Open Policy Agent (OPA)

OPA es un *framework* estándar que unifica, mediante un lenguaje programable, la implementación de **políticas de decisión**, **desacoplando** la política de control de acceso del sistema controlado (*policy decoupling*). Las políticas se escriben en un lenguaje llamado **Rego**. El servicio que necesita una decisión *delega* en OPA: le envía una *query* (cualquier valor JSON) y recibe una *decision* (otro JSON); OPA evalúa contra su *Policy* (Rego) y sus *Data* contextuales (JSON, que se actualizan o se proveen *on-demand*).

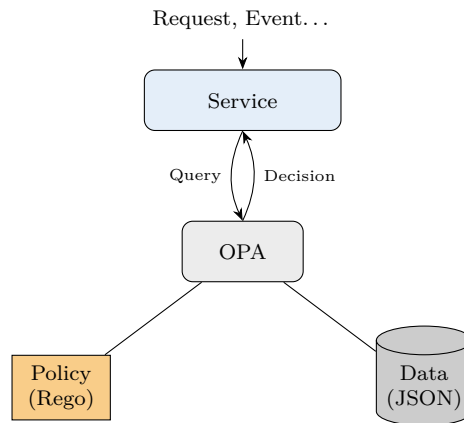


Figura 11: *Policy decoupling*: el servicio delega la decisión de acceso en OPA, que evalúa política (Rego) y datos de contexto. (Recreación de la diapositiva “Open Policy Agent”.)

Ejemplo de Rego (*admission control* en Kubernetes: solo imágenes del registro `hooli.com`):

```

package kubernetes.admission
import rego.v1

deny contains reason if {
    some container
    input_containers[container]
    not startswith(container.image, "hooli.com/")
    reason := "container image refers to illegal registry (must be hooli.com)"
}

input_containers contains container if {
    container := input.request.object.spec.containers[_]
}
    
```

14 Ejemplos reales de control de acceso

14.1 Unix / Linux file system

En el paradigma Unix, los **usuarios** son sujetos y **todo es un archivo** (monitor, memoria, disco, mouse, teclado...). Las acciones se clasifican en tres: **lectura** (*r*), **escritura** (*w*) y **ejecución** (*x*). Cada usuario pertenece a uno o más **grupos**, y todo archivo tiene un *usuario* y un *grupo* dueños. Los permisos se asignan en **tres secciones**: usuario, grupo y resto (*others*).

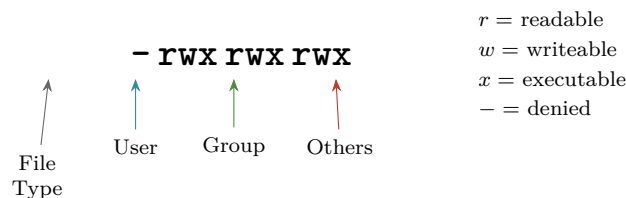


Figura 12: Permisos Unix: tipo de archivo + tres ternas **rwx** (usuario, grupo, otros). (Recreación de la diapositiva “Unix File System”.)

umask: herencia con máscara

Al crear un archivo, este hereda inicialmente los atributos del directorio. El comando **umask** (variable de entorno) le indica al SO que *saque* permisos según una máscara: p. ej. “que los archivos nuevos nunca sean ejecutables” o “que solo los pueda ver yo”, sin importar el directorio.

14.2 SELinux (Security-Enhanced Linux)

Conjunto de modificaciones al *kernel* de Linux que agrega un **control de seguridad adicional al DAC** (*hardening* de servidores). Es un conjunto de reglas que siempre responden la misma pregunta: *¿puede este sujeto acceder a este objeto?*

- Cada proceso tiene un **contexto SELinux**; todos los recursos accesibles también tienen contexto: `/var/tmp` → `t_var_tmp`; Port 80 → `t_port_www`; `/var/www/html` → `t_web_html`.
- Usa **macros** que expanden a conjuntos de reglas (limitan los *syscalls* entre combinaciones sujeto-objeto).
- Potente: **dos procesos corriendo como root (UID 0)** pueden tener restringido el acceso a los recursos del otro gracias a SELinux (p. ej. Apache `httpd_t` y MariaDB `mysqld_t` aislados entre sí).

14.3 Windows file system

El dueño de cada archivo/directorio define los permisos (**DAC**); los sujetos pueden ser cuentas o grupos. Tiene un nivel **mucho más granular** (11–13 operaciones por archivo) y —a diferencia de Unix— **herencia de permisos** que *persiste* tras la creación.

- Cada entrada puede *Allow*, *Deny* o dejar ambos en blanco (vale el heredado).
- **Deny tiene precedencia sobre Allow**; las **reglas locales** priman sobre las **heredadas**.
- Por su complejidad, Windows ofrece un **simulador de permisos efectivos** que resuelve todas las reglas y conflictos. (Históricamente, resolver permisos exigía leer muchos lugares del disco ⇒ “microcortes” al acceder a archivos.)

14.4 PostgreSQL: roles y políticas

- Usa un modelo **RBAC**: un rol puede asociarse a un usuario, a un grupo o a otro rol. El atributo `LOGIN` permite iniciar sesión. Todo objeto creado por un rol tiene a ese rol como *owner* (otro caso de **DAC**): solo *superusers* y el *owner* acceden.
- Las **políticas** crean políticas de acceso/modificación de *registros* dentro de las tablas: permiten un *need-to-know* a nivel de fila (*Row Level Security*).

```
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;
CREATE POLICY salesperson_clients ON clients
  USING (salesperson_id::text = current_user);
```

Vendedores celosos de sus contactos

Caso típico: un equipo de ventas donde cada vendedor (va a comisión) solo debe ver *sus* clientes. Con una *policy* sobre la tabla `clients` que compara `salesperson_id` con el `current_user`, una consulta sobre toda la tabla devuelve solo las filas propias (es como agregar un filtro automático). *Salvedad*: las prácticas modernas tienden a sacar la gestión de usuarios fuera de

la base de datos (la base es buena gestionando datos, no logins), así que estos *features* se ven más en el modelo “usuarios de la app = usuarios de la base”.

14.5 Squid Web Proxy

Un *web proxy* controla por dónde navegan los usuarios (y acelera/cachea). Squid implementa un sistema de **reglas ordenadas (MAC)**; “mal llamado ACL”) para permitir o bloquear sitios.

- Filtra por dominio, protocolo, URL, *headers* del requerimiento y **listas negras**.
- En la facultad, los sitios de *streaming* se bloquean con esta tecnología.
- **Blacklists dinámicas**: iniciativas que recolectan IPs de origen de ataques, metodologías y *malware* usado; Squid puede consultarlas para bloquear accesos.

14.6 AWS (IAM)

AWS (más de 1000 servicios) combina **dos mecanismos**:

Identity-based policies	Resource-based policies
Asociadas al <i>Principal</i> (MAC)	Asociadas al <i>Recurso</i> (DAC)
Reglas asociadas a la cuenta/usuario; las define el administrador . Cubren “lo grueso” de la infraestructura (subredes, firewalls).	Quien <i>crea</i> el recurso define quién accede (p. ej. una <i>S3 bucket policy</i>). Más ágil para los equipos de desarrollo.

14.7 API gateway

Servicio que se despliega *delante* de las APIs para una capa extra de seguridad.

- Controles **basados en reglas (MAC)** por URL, dominio, *headers*, método.
- Reglas **por contexto**: máxima cantidad de requerimientos por segundo, ancho de banda; si se superan los umbrales, bloquea nuevas conexiones del mismo IP o *SessionID*.

15 OAuth 2.0 (Open Authorization)

OAuth es un estándar de **autorización** ampliamente usado para que las aplicaciones accedan a recursos de una plataforma *en nombre del usuario* **sin que este comparta sus credenciales** con la aplicación. Está detrás de “Iniciar sesión con Google/Meta”. Funciona emitiendo **tokens de acceso temporales y específicos**; **OAuth 2.0** es la versión moderna.

El problema que resuelve

La forma “fea” sería darle a la app mi usuario y contraseña del servicio: eso le daría poder para borrar la cuenta, cambiar la clave, etc., y mis credenciales quedarían en una base que no controlo. OAuth nació (Twitter) para que un usuario pueda *delegar* programáticamente a una aplicación la capacidad de ejecutar cosas en su nombre, *sin entregar credenciales*.

15.1 El flujo de autorización

Intervienen el *Resource Owner* (usuario), la *Application*, el *Authorization Server* y el *Resource Server*.

Puntos clave del flujo:

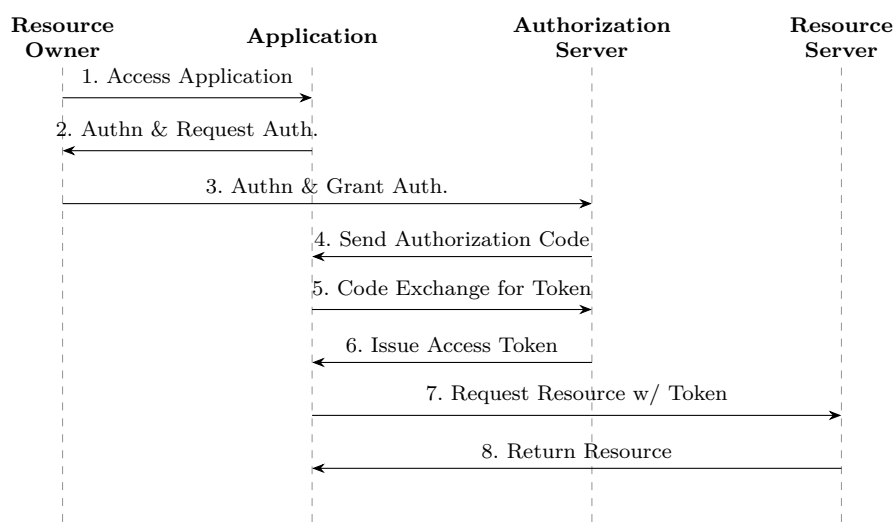


Figura 13: Flujo de *authorization code*: el usuario autoriza, la app obtiene un *code*, lo intercambia por un *access token* y con él pide el recurso. (*Recreación de la diapositiva “OAuth 2.0”.*)

- El usuario nunca comparte con la app las credenciales del servicio; **delega** el control de acceso al *dueño del recurso*, que decide si responde.
- El *access token* viaja en un *header* (típicamente **Bearer**); el servidor lo valida en cada requerimiento. El usuario puede **revocar** el acceso.

15.2 Refresh Token

Los *access tokens* **no** duran para siempre (de una hora a unos días). Aparecen **dos tokens**: *access* y *refresh*.

- Cuando el recurso responde “token inválido”, el cliente envía el *refresh token* y obtiene —sin re-preguntar al usuario— un *access* y un *refresh* nuevos.
- El *refresh token* se usa **una sola vez y muere**. Esto permite **recuperarse ante un robo**: si un atacante roba ambos tokens, en cuanto uno de los dos (atacante o app legítima) usa el *refresh*, *el otro pierde acceso*.
- Así se sostiene la “sesión infinita” (Gmail, Mercado Libre...) sin el riesgo de un token eterno que, si se roba, sería *game over*.

15.3 Scope (alcance) — un mecanismo DAC

El **scope** son los permisos específicos que una app solicita para acceder a los datos del usuario; definen el **nivel de acceso** que tendrá el token.

- Al pedir autorización, la app declara qué *scopes* quiere; eso genera la pantalla de consentimiento (“*Get Together* quiere acceder a tu email y a tu calendario”).
- El token emitido se **limita a los scopes otorgados**; cada servicio que lo recibe verifica que sea válido y que contenga su propio scope.

OAuth es dual

La clase siguiente (Autenticación) retoma OAuth: hoy es un protocolo **dual** que cumple dos funciones de seguridad. Aquí vimos la primera (*autorización*); la otra (*autenticación*, vía OpenID Connect) requiere un poco más de teoría.

Lectura recomendada

Capítulo 4 · Capítulo 5 (5.1–5.4) · Capítulo 6 (6.1–6.2)
Capítulo 7 (7.1) · Capítulo 8 (8.1)

Computer Security: Art and Science

Matt Bishop

Los temas de políticas y control de acceso corresponden a los capítulos 4 al 8 del Bishop, con más ejemplos. Mención especial: el modelo de **capabilities** (un mecanismo dual de las ACL, muy útil en sistemas distribuidos a escala gigantesca). Los papers originales de los modelos: [Bell76.pdf](#), [Biba75.pdf](#) y [A_Comparison_of_commercial_and_military.pdf](#).

Apunte integral de la Clase 6 — Políticas de seguridad y Control de acceso · Criptografía y Seguridad, ITBA.

Síntesis de teoría (transcripción + diapositivas) y práctica (diapositivas).